

ClotCAD Cheatsheet dev

ClotCAD

May 28, 2026

1 3D Primitives

```
make-box(dx, dy, dz)
make-cylinder(radius, height)
make-sphere(radius)
make-cone(r1, r2, height)
make-torus(major-radius, minor-radius)
```

2 Sweeps

```
make-prism(shape, dx, dy, dz)
make-revol(shape, ax, ay, az, angle-deg)
```

3 3D Text

```
make-3d-text(string, ..options)
```

Options

```
:font - name string or auto-fallback
:size - font size (default 10)
:thickness - extrusion depth (default 5)
:plane - :xy, :xz(default), :yz, face,
frame
:h-align - :left, :center(default), :right
:v-align - :bottom, :center(default), :top
```

4 Boolean Operations

```
cut(shape, &rest others)
fuse(shape, &rest others)
common(shape, &rest others)
section(shape, &rest others)
```

5 Transformations

```
translate(shape, dx, dy, dz)
rotate(shape, ax, ay, az, angle-deg)
```

6 2D Geometry

Points & Vectors

```
make-pnt2d(x, y)
make-vec2d(x, y)
make-dir2d(x, y)
```

Lines & Circles

```
make-line2d(x, y, dx, dy)
make-circle2d(x, y, radius)
```

Edges

```
make-edge(x1, y1, x2, y2)
make-edge-3d(x1, y1, z1, x2, y2, z2)
make-circle-edge(x, y, radius)
make-circular-arc(x1, y1, x2, y2, x3, y3)
```

Wires & Faces

```
make-wire(&rest edges)
make-face(wire)
make-face-on-plane(wire, ox, oy, oz, nx, ny, nz)
```

7 Object Display Management

```
display(name, shape, ..options)
clear-all()
def(name, shape-form)
show(&rest names)
hide(&rest names)
toggle(&rest names)
show-defs(on)
toggle-defs()
resolve-shape(designator)
```

8 Selection

```
select(&rest designators)
deselect(&rest designators)
```

```
clear-selection()
selected-shapes()
apply-selection-schemes(..schemes)
```

9 Compounds & Assemblies

```
make-compound(shapes)
add-to-compound(compound, shape)
make-part(shape, ..options)
make-assembly(..options)
```

10 Parametric DSL

Macros

```
defmodel(name, (params), ..body)
with-params((&rest bindings), ..body)
```

Functions

```
param(key)
model-ref(name)
model-color(name)
model-display-name(name)
model-layer(name)
```

Variables

```
*params* - parameter plist
*model-registry* - DAG registry
```

Mutation

```
set-param!(key, value)
set-params!(&rest key-values)
```

Examples

```
(defmodel base-plate (w d)
  (make-box (param :w) 5 (param :d)))

(defmodel assembly (h)
  (fuse (model-ref :base-plate)
        (make-cylinder 5 (param :h))))

(defmodel bracket (w h d r)
  (let ((box (make-box (param :w) (param :h)
                        (param :d))))
    (hole (make-cylinder (param :r)
```

```
(param :d)))
  (cut box (translate hole
    (/ (param :w) 2) (/ (param :h) 2) 0)))

(set-params! :w 60 :d 40 :h 20 :r 3)

(display :assy (model-ref :assembly))

(with-params (:w 80 :h 30)
  (def :br (model-ref :bracket)))
(show :br)

(fit-view)
```

11 View Controls

```
set-view(orientation)
current-view()
fit-view()
set-view-aa(enable)
```

Toggles

```
show-grid(&optional show)
toggle-grid()
show-axis(&optional show)
toggle-axis()
show-viewcube(&optional show)
toggle-viewcube()
show-viewcube-axes(&optional show)
toggle-viewcube-axes()
```

Orientation

```
:top :bottom :front :back
:left :right :iso
```

12 Dock Panels

```
show-repl(&optional show)
toggle-repl()
show-scene-tree(&optional show)
toggle-scene-tree()
```

13 Theme

```
apply-theme(mode, ..options)
theme-dark(&optional accent)
theme-light(&optional accent)
theme-auto(&optional accent)
set-accent(color-hex)
set-font-size(size)
```

14 File I/O

```
write-step(shape, filename)
read-step(filename)
write-stl(shape, filename, ..options)
```

```
read-stl(filename)
write-dag-models-to-step(path)
read-step-into-dag(path)
help()
```

15 Threading Macros

```
->> (thread-last) inserts result at end:
(->> (make-box 10 10 10) (display :box))

as-> lets you place var explicitly:
(as-> (make-box 10 10 10) v (display :box v))

-> inserts result at start:
(-> (make-box 10 20 30) (translate 5 0 0))
```

16 Introspection

```
doc(name)
browse(pattern, ..options)
```

Prints docstring & arglist for any symbol.
Searches API symbols by substring.

17 REPL & Import

```
cancel-import()
replay-speed(ms)
result-export(flag)
export-repl-history(path)
set-repl-history-key(modifier)
set-repl-submit-key(modifier)
```